
TalentMatch

Plateforme intelligente de matching CV / Offres d'emploi

Rapport Technique

Choix technologiques, architecture et implémentation

Réalisé par :	Youssef Gara
Entreprise :	3S Group — Tunis
Encadrante société :	Mme Sirine Nasri
Encadrante académique :	Mme Loujayne Bouzarti (Esprit)
Période :	Février 2026 — Août 2026
Date :	Juin 2026

Table des matières

1	Contexte et objectif du projet	3
1.1	Problématique	3
1.2	Objectif — TalentMatch	3
2	Architecture générale du système	4
2.1	Vue d'ensemble	4
2.2	Structure du code	4
3	Choix technologiques et justifications	5
3.1	Backend — FastAPI	5
3.2	Base de données — PostgreSQL	5
3.3	Frontend — React 18 + Vite	6
3.4	ORM — SQLAlchemy 2 + Alembic	6
4	Pipeline d'ingestion des CVs	7
4.1	Étape 1 — Extraction du texte	7
4.2	Étape 2 — Parsing NLP	7
4.3	Étape 3 — Stockage structuré	8
5	Moteur de Matching IA	9
5.1	Vue d'ensemble du scoring	9
5.2	Composant 1 — BGE-M3 (Bi-encoder)	9
5.3	Composant 2 — Cross-Encoder (Reranker)	10
5.4	Composant 3 — MLP Fusion (10 features)	10
5.4.1	Architecture du MLP	10
5.4.2	Les 10 features d'entrée	11
5.4.3	Entraînement du MLP	12
5.5	Domaine ISCO-08 — EscoService	12
5.6	Score final — Pipeline complet	12
6	Améliorations avancées du moteur de matching	14
6.1	Qualité des données — Ontologie de synonymes (A1)	14
6.2	Qualité des données — Extraction multi-domaine (A2, A3)	14
6.3	Logique métier — Compétence centrale du titre (Phase 1)	15
6.4	Logique métier — Pondération par criticité (Phase 2)	15
6.5	Logique métier — Recalibration par type d'offre (Phase 3)	15

6.6	Apprentissage humain dans la boucle	16
6.7	Évaluation systématique du modèle	16
7	Sécurité et conformité RGPD	18
7.1	Authentification JWT	18
7.2	Rôles et contrôle d'accès	18
7.3	Conformité RGPD	18
8	Interface utilisateur (Frontend)	19
8.1	Architecture React	19
8.2	Pages principales	19
8.3	Rapport de matching — MatchReport	19
9	Méthodologie — Agile Scrum	20
9.1	Organisation des sprints	20
9.2	Outils utilisés	20
10	Avancement actuel — Sprint 4	21
10.1	Fonctionnalités livrées	21
10.2	Démo — Scénario de test	21
11	Perspectives	22
	Conclusion	23

Chapitre 1

Contexte et objectif du projet

1.1 Problématique

Le recrutement moderne souffre d'un paradoxe : les entreprises reçoivent un volume croissant de candidatures, mais les outils existants (LinkedIn, Indeed, ANAPEC) se limitent à une recherche par **mots-clés**, sans comprendre le sens réel des profils.

Chiffre clé

Un recruteur consulte un CV en moyenne **23 secondes**. Sans outil intelligent, de nombreux bons profils sont ignorés ou mal classés.

Les problèmes concrets identifiés :

- Faux positifs : un profil qui cite un mot-clé sans maîtriser le domaine.
- Bons profils ignorés : un expert qui formule ses compétences différemment.
- Pas de compréhension du **contexte sémantique** ni de la trajectoire professionnelle.
- Absence d'explication : le recruteur ne sait pas *pourquoi* un candidat est classé 1^{er}.

1.2 Objectif — TalentMatch

TalentMatch est une plateforme web complète qui automatise le **matching CV / offres d'emploi** via un pipeline IA :

1. **Extraction** — lire un CV (PDF, DOCX, scan) et en extraire le texte brut.
2. **Parsing NLP** — structurer le texte en JSON (compétences, expériences, formations).
3. **Scoring IA** — calculer un score de pertinence 0–100% par modèle d'embeddings.
4. **Classement et explication** — classer les candidats et afficher un rapport détaillé.

Chapitre 2

Architecture générale du système

2.1 Vue d'ensemble

Le système suit une architecture **client-serveur découplée** :

Couche	Technologie	Rôle
Frontend	React 18 + Vite	Interface utilisateur (SPA)
API	FastAPI (Python)	Logique métier, endpoints REST
IA / NLP	spaCy, BGE-M3, MLP	Parsing et matching
Base de données	PostgreSQL 16	Stockage persistant
Auth	JWT (python-jose)	Sécurité des accès
Email	SMTP Gmail	Notifications
Déploiement	Railway + Docker	Cloud

2.2 Structure du code

Arborescence principale — projet 3s-talentmatch/

```
backend/ app/ main.py <- Point d'entrée FastAPI (lifespan,
CORS, routes) routes/ <- Endpoints REST (auth, cv, matching,
job_offers...)models/ <- TablesSQLAlchemy(ORM)schemas/ <-
ValidationsPydanticsservices/extraction/ <- LecturePDF/DOCX/OCRnlp/ <-
PipelineNLP(spaCy)matching/ <- Moteurdematchingprincipalmatching_sandbox/ <-
ScoreursIA(BGE-M3,MLP)ai_summary/ <- RsumIAduprofil(Claude)scripts/ <-
Scriptsd'entrainementetdeseed
frontend/ src/ pages/ <- Pages React (Dashboard, OfferDetail...)
components/ <- Composants réutilisables (MatchReport...) services/api.js
<- Instance Axios centralisée
```

Chapitre 3

Choix technologiques et justifications

3.1 Backend — FastAPI

Pourquoi FastAPI et pas Flask ou Django ?

- **Async natif** : FastAPI est basé sur ASGI (Starlette), ce qui permet de gérer plusieurs requêtes simultanées sans bloquer — crucial pour les appels aux modèles IA qui sont lents.
- **Documentation automatique** : Swagger UI sur `/docs` et ReDoc sur `/redoc` générés automatiquement depuis les types Python.
- **Validation automatique** : les schémas Pydantic valident les entrées/sorties.
- **Flask** : trop bas niveau, pas de validation intégrée, sync par défaut.
- **Django** : monolithique, overhead inutile pour une API pure.

Fichier clé

`backend/app/main.py` — point d'entrée FastAPI. Le **lifespan handler** (lignes 1–50) pré-chauffe le modèle BGE-M3 au démarrage pour éviter la latence au premier appel de matching.

3.2 Base de données — PostgreSQL

Pourquoi PostgreSQL et pas MySQL ou MongoDB ?

- **ACID** : transactions fiables — important pour les opérations de matching qui mettent à jour plusieurs tables en même temps.
- **JSON natif** : le champ `parsed_data` (profil du candidat structuré) est stocké en JSONB, ce qui permet des requêtes sur les sous-champs.
- **MongoDB** : schéma trop flexible, difficile à maintenir avec des relations (candidat offre match).
- Port utilisé : **5433** (non-standard pour éviter les conflits avec d'autres instances locales).

Tables principales :

Table	Description
<code>users</code>	Comptes (candidat / recruteur / admin), auth JWT + OAuth
<code>candidates</code>	Profil structuré du candidat + <code>parsed_data</code> JSONB
<code>cv_documents</code>	Fichier CV uploadé (référence vers stockage)
<code>job_offers</code>	Offres d'emploi (titre, description, compétences requises)
<code>matches</code>	Score de matching candidatoffre (0–100%)
<code>notifications</code>	Alertes email (nouvelles candidatures, statut)
<code>access_logs</code>	Journal RGPD — chaque consultation de données personnelles

Fichiers clés

[backend/app/models/candidate.py](#) — modèle Candidate (`parsed_data` JSONB).
[backend/app/models/match.py](#) — table des scores de matching.
[backend/app/models/access_log.py](#) — journal RGPD.
[backend/alembic/versions/](#) — 9 fichiers de migration.

3.3 Frontend — React 18 + Vite

Pourquoi React et pas Vue ou Angular ?

- **Composants réutilisables** : le composant `MatchReport` est utilisé à la fois dans le dashboard recruteur et la page candidat.
- **Hooks** (`useState`, `useEffect`, `useContext`) : gestion d'état simple sans Redux.
- **Vite** : serveur de développement ultra-rapide (HMR instantané).
- **Angular** : trop verbose, TypeScript obligatoire, courbe d'apprentissage.

Point d'attention — Instance Axios centralisée

[frontend/src/services/api.js](#) — **toujours** importer depuis ce fichier. Il injecte automatiquement le JWT dans chaque requête et redirige vers `/login` en cas de 401. Contourner ce fichier casserait l'authentification.

3.4 ORM — SQLAlchemy 2 + Alembic

- **SQLAlchemy 2** : ORM — on écrit du Python, pas du SQL brut. Préviens les injections SQL.
- **Alembic** : versioning du schéma de BDD. Chaque modification de modèle génère un fichier de migration dans [backend/alembic/versions/](#).

Chapitre 4

Pipeline d'ingestion des CVs

Le pipeline complet se déclenche sur POST `/api/upload-cv` ([backend/app/routes/cv.py](#)).

4.1 Étape 1 — Extraction du texte

Le module [backend/app/services/extraction/](#) dispatche selon le type de fichier :

Type	Outil	Fichier
PDF textuel	PyMuPDF / pypdf	<code>pdf_extractor.py</code>
DOCX (Word)	python-docx	<code>word_extractor.py</code>
Image / PDF scanné	EasyOCR	<code>ocr_extractor.py</code>

Pourquoi EasyOCR et pas Tesseract ?

- EasyOCR utilise du **Deep Learning** (CRAFT + CRNN) — meilleure précision sur les documents bruités, inclinés ou avec des polices non standard.
- Tesseract est basé sur de l'OCR classique, plus fragile sur les CVs scannés de mauvaise qualité.
- EasyOCR supporte le français et l'arabe nativement.

4.2 Étape 2 — Parsing NLP

L'orchestrateur [backend/app/services/nlp/nlp_parser.py](#) appelle les extracteurs dans cet ordre :

1. **ContactExtractor** — nom, email, téléphone, adresse.
2. **EntityExtractor** — entités nommées via spaCy NER. Optionnel : **HFCamembertNameExtractor** si `HF_ENABLE_CAMEMBERT_NAME=1`.
3. **SkillsExtractor** — compétences techniques et soft skills. Fichier : [backend/app/services/nlp/](#)
4. **FormationExtractor** — diplômes, établissements, années. Fichier : [backend/app/services/nl](#)
5. **ExperienceExtractor** — postes, entreprises, durées. Fichier : [backend/app/services/nlp/exp](#)
6. **Langues** — langues détectées automatiquement (`_extract_langues`).

Pourquoi spaCy fr_core_news_md et pas NLTK ?

- spaCy propose une **pipeline NLP complète** (tokenisation, POS, dépendances, NER) en un seul modèle.
- Le modèle `fr_core_news_md` est entraîné sur du français journalistique et institutionnel — bien adapté aux CVs.
- NLTK est un toolkit académique sans modèle NER français de qualité.
- **Important** : le modèle n'est pas installable via pip. Il faut : `python -m spacy download fr_core_news_md`

4.3 Étape 3 — Stockage structuré

Le résultat du parsing est stocké dans `Candidate.parsed_data` (champ JSONB PostgreSQL). Sa structure est définie dans [backend/app/schemas/cv_data.py](#).

Exemple de structure :

```
1 {  
2   "identite": {"nom_complet": "...", "email": "..."},  
3   "competences": {"techniques": ["React", "Python"], "soft": ["  
4     Leadership"]},  
5   "experiences": [{"poste": "Dev_React", "duree_mois": 24,  
6     ...}],  
7   "formations": [{"diplome": "Licence_Info", "etablissement": "  
8     Esprit"}],  
9   "secteur_detecte": {"secteur": "informatique", "label": "  
10     D veloppeur"},  
11   "metadata": {"spacy_fallback": false, "version": "2.1"}  
12 }
```

Chapitre 5

Moteur de Matching IA

5.1 Vue d'ensemble du scoring

Le score final (0–100%) est calculé par le scoreur principal `backend/app/services/matching_sandbox` via une architecture à **3 niveaux** :

Niveau	Composant	Rôle
1	BGE-M3 (bi-encoder)	Embeddings sémantiques
2	Cross-Encoder (reranker)	Score sémantique précis
3	MLP Fusion (7 features)	Score final combiné

5.2 Composant 1 — BGE-M3 (Bi-encoder)

Qu'est-ce que BGE-M3 ?

BGE-M3 (**BAAI/bge-m3**) est un modèle de *sentence embeddings* multilingue développé par le Beijing Academy of Artificial Intelligence.

- **568 millions** de paramètres, vecteurs de dimension **1024**.
- Supporte **100+ langues** — dont français, arabe et anglais en même temps.
- Basé sur l'architecture **XLM-RoBERTa** fine-tuné pour la similarité sémantique.
- Il encode séparément le CV et l'offre, puis calcule la **similarité cosinus** entre les deux vecteurs.

Pourquoi BGE-M3 et pas CamemBERT ?

- **CamemBERT** est monolingue (français uniquement). Les CVs tunisiens contiennent souvent de l'anglais, du français et parfois de l'arabe.
- BGE-M3 est spécifiquement entraîné pour la **recherche sémantique** et la **similarité de documents**, pas juste pour le NER ou la classification.
- CamemBERT était utilisé dans les versions antérieures (Sprint 2-3). Le passage à BGE-M3 au Sprint 4 a amélioré les scores de matching.

Localisation dans le code

```
Fichier : backend/app/services/matching_sandbox/bert_scorer.py Ligne 734 : E
MBEDDER_MODEL = "BAAI/bge-m3" Ligne 758 : class BERTMatchingScorer :
- > methode_encode() : calcule les embeddings - > methode_score() :
retourne la similarité cosinus
```

5.3 Composant 2 — Cross-Encoder (Reranker)**Qu'est-ce qu'un Cross-Encoder ?**

Contrairement au bi-encoder (BGE-M3 encode séparément les deux textes), le Cross-Encoder (**BAAI/bge-reranker-v2-m3**) prend la **paire (offre, CV) comme entrée unique**.

- Il peut ainsi capturer des **interactions fines** entre les deux textes (un mot de l'offre qui répond à une phrase du CV).
- Résultat : un score scalaire unique, plus précis que la similarité cosinus.
- **Limitation** : beaucoup plus lent (pas de cache d'embeddings possible). C'est pourquoi il est utilisé en *reranker* après un premier filtre BGE-M3.

Localisation dans le code

```
Fichier : backend/app/services/matching_sandbox/bert_scorer.py Ligne 737 : R
ERANKER_MODEL = "BAAI/bge-reranker-v2-m3" Ligne 794 :
def load_reranker(self) : - > CrossEncoderChargenLazyLoading(aupremierappel) - >
max_length = 512 pour correspondre au modèle
```

5.4 Composant 3 — MLP Fusion (10 features)**5.4.1 Architecture du MLP**

Le MLP (*Multi-Layer Perceptron*) est un réseau de neurones simple entraîné **spécifiquement sur les données du projet**. La version actuelle (v3b1) prend **10 features** en entrée, contre 7 dans la première version : trois signaux métier supplémentaires ont été ajoutés (compétence du titre, criticité des compétences, type d'offre) — détaillés au chapitre 6.

Architecture — `backend/app/services/matching_sandbox/bert_scorer.py`
(classe `_ScoringMLPv3b1`)

10 features $\rightarrow$ 64 neurones $\rightarrow$ 32 neurones $\rightarrow$ 1 score

Couches : Linéaire + BatchNorm + ReLU + Dropout 0.20, puis Linéaire + ReLU + Dropout 0.10, puis Linéaire + Sigmoid (score entre 0 et 1).

5.4.2 Les 10 features d'entrée

#	Feature	Description
1	sem_sim	Similarité sémantique BGE-M3 (bi-encoder)
2	skill_rate_w	Ratio compétences pondéré par criticité (Phase 2)
3	exp_score	Score d'expérience (années vs. requis)
4	form_score	Score de formation, intégrant le domaine d'études (A3)
5	sem_v2	Score du Cross-Encoder (reranker) sur la paire
6	edu_dom_compat	Compatibilité du domaine d'études candidat / offre
7	exp_dom_ratio	Ratio d'expérience dans le domaine de l'offre
8	title_skill_match	1.0 si la compétence du titre est présente, 0.0 sinon (Phase 1)
9	crit_skill_ratio	Ratio de compétences critiques matchées (Phase 2)
10	offer_type_enc	Type d'offre encodé : 0=junior / 0.25=management / 0.5=tech / 1=senior (Phase 3)

Pourquoi un MLP et pas une formule pondérée fixe ?

- Une formule fixe (ex : 40% sémantique + 30% skills + 30% expérience) est **rigide** et ne s'adapte pas aux nuances métier.
- Le MLP **apprend les poids optimaux** depuis les données d'entraînement.
- Il peut apprendre des interactions non-linéaires (ex : un candidat sans diplôme requis mais avec 10 ans d'expérience est quand même pertinent).

Méthode unique `_compute_mlp_features()` — zéro décalage train/prod

Le vecteur de 10 features est produit par **une seule méthode**, utilisée à la fois pour l'inférence en production et pour la génération du dataset d'entraînement. Cela garantit que le modèle apprend exactement sur les valeurs qu'il rencontrera en production (pas de *train/serving skew*).

5.4.3 Entraînement du MLP

Fichiers d'entraînement (version v3b1)

- `backend/scripts/generate_mlp_dataset_v3b1.py` → génère ~ 360 paires (candidat, offre) couvrant 8 domaines ISCO et 3 types d'offres, dont 30% de cas hors-domaine. Encodage BGE-M3 en *batch* (génération en moins de 6 secondes).
- `backend/scripts/train_mlp_v3b1.py` → entraîne le MLP $10 \rightarrow 64 \rightarrow 32 \rightarrow 1$ (split 80/10/10, early stopping). Sauvegarde les poids dans `data/models/fusion_mlp/fusion_mlp_v3b1.pt`. Résultats : précision binaire 97%, MAE 5,9%.
- Le modèle entraîné est chargé au démarrage du serveur via le lifespan handler (`backend/app/main.py`). Influence du MLP dans le score final : 40%.

5.5 Domaine ISCO-08 — EscoService

Qu'est-ce que l'ISCO-08 ?

L'ISCO-08 (*International Standard Classification of Occupations 2008*) est la classification internationale des métiers de l'OIT (Organisation Internationale du Travail). Elle couvre **53 domaines professionnels**.

Rôle dans TalentMatch : avant de calculer le score, le système détecte le domaine professionnel du CV et de l'offre via `backend/app/services/matching_sandbox/esco_service.py`.

- **Même domaine** (ex : Dev React Offre React) → pas de cap, score libre.
- **Domaines adjacents** (ex : Data Science Dev Backend) → cap à 0.65 (65%).
- **Domaines différents** (ex : Médecin Offre Dev) → cap à 0.35 (35%).

Localisation

```
Fichier : backend/app/services/matching_sandbox/esco_service.py
Données : data/esco/isco08_fr.json (53 groupes, keywords, skills_fr)
Méthode : EscoService.get_score_cap(domain1, domain2) → float|None
```

5.6 Score final — Pipeline complet

Étape	Opération	Résultat
1	BGE-M3 encode CV + Offre séparément	Vecteurs 1024-dim
2	Similarité cosinus entre les deux vecteurs	Score sémantique brut
3	Cross-Encoder (CV, Offre) en entrée unique	Score sémantique raffiné
4	Calcul des 10 features (<code>_compute_mlp_features</code>)	Vecteur 10 valeurs
5	MLP Fusion (40%) + formule calibrée (60%)	Score 0–1
6	Multiplicateur de domaine ISCO-08 + cap titre (Phase 1)	Score ajusté
7	Multiplication $\times 100$	Score % affiché

Chapitre 6

Améliorations avancées du moteur de matching

Ce chapitre décrit les améliorations apportées au moteur après la première version. Elles se répartissent en deux familles : la **qualité des données d'entrée** (phases A1–A3) et la **logique métier de scoring** (phases 1–3), suivies de la boucle d'apprentissage humain et de l'évaluation systématique du modèle.

6.1 Qualité des données — Ontologie de synonymes (A1)

Le problème résolu

La première version reconnaissait mal les synonymes de compétences. « ReactJS », « React.js » et « React » n'étaient pas reliés ; « recruiting » (anglais) ne matchait pas « recrutement ». Pire, le dictionnaire généré automatiquement contenait du bruit (ex : « react » → « nuclear energy »).

Une ontologie propre a été construite dans `data/skills/skills_aliases.json` (5 047 **compétences** normalisées, FR + EN, multi-domaine). Elle fusionne : un dictionnaire IT curé manuellement, un dictionnaire métier (Finance, RH, Santé, Droit, BTP, Logistique...), et les labels ESCO officiels. La normalisation (minuscules, suppression des accents) est appliquée **avant** le calcul du ratio de compétences.

Génération

Script : `scripts/build_skills_aliases.py` Sortie : `data/skills/skills_aliases.json` (5047 entrées) Test :
"ReactJS" == "React.js" == "React" → match(8/8 validations)

6.2 Qualité des données — Extraction multi-domaine (A2, A3)

A2 — Compétences non-IT. L'extracteur de compétences (`skills_extractor.py`) a été corrigé pour gérer les compétences composées (« contrôle de gestion », « gestion prévisionnelle des emplois ») et les domaines non informatiques. Validation : **100%** de rappel sur des CV de référence en Comptabilité, RH et Santé (38/38 compétences extraites).

A3 — Domaine d'études. L'extracteur de formations (`formation_extractor.py`) ne se limite plus au niveau du diplôme (Bac+5) mais en extrait aussi le **domaine** (informatique, finance, droit...). Ainsi « Master Informatique » et « Master Finance », de même niveau, sont désormais distingués pour le scoring de la formation.

6.3 Logique métier — Compétence centrale du titre (Phase 1)

Le problème résolu

Un développeur Java avec 4 ans d'expérience obtenait un meilleur score qu'un développeur Python junior pour un poste « Développeur Python » — alors que Python est la compétence **obligatoire** indiquée dans le titre.

Le système détecte la compétence centrale du titre de manière **sémantique** (via BGE-M3, sans liste codée en dur) : il compare le titre à chaque compétence requise et retient la plus proche. Si le candidat ne possède pas cette compétence **et** qu'il est dans le même domaine, son score est plafonné ($\times 0,50$). Résultat : le développeur Java passe de 69% à 34% pour un poste Python.

6.4 Logique métier — Pondération par criticité (Phase 2)

Toutes les compétences n'ont pas la même importance. Le système les classe en trois niveaux de poids :

Niveau	Poids	Exemple (poste Dev Python)
Critique (compétence du titre)	$\times 3$	Python
Requise (métier)	$\times 2$	FastAPI, PostgreSQL
Générique (outil transversal)	$\times 1$	Git, Docker, Excel

Le ratio de compétences est alors normalisé par la somme des poids, ce qui donne plus de valeur aux compétences réellement discriminantes.

6.5 Logique métier — Recalibration par type d'offre (Phase 3)

Les poids des quatre dimensions (compétences, sémantique, expérience, formation) ne sont plus fixes : ils s'adaptent au type d'offre détecté.

Type d'offre	Skills	Sém.	Exp.	Form.
Tech spécialisé	55%	20%	15%	10%
Senior 5+ ans	40%	20%	30%	10%
Stage / Junior	45%	25%	5%	25%
Management / RH	35%	25%	25%	15%

Le seuil de décision « Entretien recommandé » s'ajuste lui aussi (65% pour un poste junior, 70% pour un senior).

6.6 Apprentissage humain dans la boucle

Pour ne pas apprendre uniquement sur une formule (raisonnement circulaire), un outil d'annotation ([scripts/label_pairs.py](#)) permet à un recruteur d'attribuer un label réel (0 = mauvais, 0,5 = moyen, 1 = bon) à des paires (offre, CV) sélectionnées de façon stratifiée. Ces labels sont stockés dans [data/human_labels.csv](#) et mélangés au dataset formule lors du réentraînement, avec un **poids double** pour la vérité terrain humaine ([train_mlp_human_labels.py](#)). Un premier lot de 40 annotations a confirmé un accord de 39/40 avec le modèle, validant sa calibration.

6.7 Évaluation systématique du modèle

Un *benchmark* ([scripts/benchmark_model.py](#)) mesure les performances du modèle par **catégorie** de situation, pour cartographier précisément ses forces et faiblesses.

Catégorie testée	Résultat	Statut
Match parfait (même domaine)	3/3	Fort
Rejet hors-domaine total	3/3	Fort
Domaines adjacents (zone grise)	2/2	Fort
Synonymes EN/FR (<i>aliases</i>)	2/2	Fort
Cap compétence titre (Phase 1)	2/2	Fort
Surqualification	1/1	Fort
Sous-qualification expérience	2/2	Fort
Discrimination junior (exp. faible)	1/2	Moyen
Score global	16/17	94%

Force principale. Le modèle distingue très nettement les bons profils (85–91%) des profils hors-domaine (5–18%), y compris sur les métiers non informatiques, et reconnaît les synonymes multilingues.

Point faible identifié. Sur les offres junior/stage, le poids élevé de la formation (25%) peut gonfler le score d'un candidat ayant le bon diplôme mais peu de compétences. Cet axe d'amélioration est documenté pour une itération future.

Tests de non-régression

En complément du benchmark, deux suites garantissent la stabilité : `test_matching_reel.py` (15 cas de scoring, dont 6 multi-domaine) et `test_ranking.py` (classement validé par corrélation de Spearman sur 3 scénarios : $\rho = 0,90$ à $1,00$).

Chapitre 7

Sécurité et conformité RGPD

7.1 Authentification JWT

- **JSON Web Token (JWT)** : standard RFC 7519. Un token signé est émis à la connexion et doit être envoyé dans chaque requête (header **Authorization: Bearer <token>**).
- Le token contient l’ID utilisateur et le rôle (candidat / recruteur / admin).
- Expiration configurable via `ACCESS_TOKEN_EXPIRE_MINUTES` dans `backend/.env`.
- Implémentation : `backend/app/services/auth_service.py`

Pourquoi JWT et pas sessions classiques ?

- **Stateless** : le serveur n’a pas besoin de stocker les sessions en BDD. Scalable horizontalement (plusieurs instances du serveur).
- **Déployable sur Railway** sans partage de session entre instances.

7.2 Rôles et contrôle d’accès

Trois rôles définis dans `backend/app/dependencies.py` :

Rôle	Accès
Candidat	Upload CV, voir ses candidatures, liste des offres
Recruteur	Gérer ses offres, voir les résultats de matching
Admin	Tout + gestion utilisateurs + logs RGPD

7.3 Conformité RGPD

- **Consentement explicite** : lors de la candidature, le candidat coche une case RGPD. Visible sur la capture *postuleCv.png*.
- **Journal d’accès** : table `access_logs` (`backend/app/models/access_log.py`) — chaque consultation de données personnelles est enregistrée (qui, quand, quoi).
- **Droit à l’effacement** : endpoint dédié de suppression complète.
- **Conservation limitée** : suppression automatique après 2 ans d’inactivité.

Chapitre 8

Interface utilisateur (Frontend)

8.1 Architecture React

Fichier central — `frontend/src/services/api.js`

Toutes les requêtes API passent par cette instance Axios unique. Elle injecte automatiquement le JWT, gère le timeout (8s) et redirige vers `/login` si le serveur répond 401. **Ne jamais appeler Axios directement** dans un composant — toujours passer par ce fichier.

8.2 Pages principales

Page / Fichier	Description
<code>frontend/src/pages/Dashboard.jsx</code>	Tableau de bord recruteur
<code>frontend/src/pages/OffersList.jsx</code>	Liste des offres (côté candidat)
<code>frontend/src/pages/OfferDetail.jsx</code>	Détail + bouton Postuler
<code>frontend/src/pages/OfferNew.jsx</code>	Création d'une nouvelle offre
<code>frontend/src/components/MatchReport.jsx</code>	Rapport de matching IA détaillé

8.3 Rapport de matching — MatchReport

Le composant `frontend/src/components/MatchReport.jsx` affiche pour chaque candidat :

- Score global (0–100%) avec code couleur.
- Décomposition par dimension (Compétences, Expérience, Formation, Sémantique).
- Compétences présentes vs manquantes.
- Bouton « Analyse IA (Mistral) » → rapport narratif généré par LLM.

Chapitre 9

Méthodologie — Agile Scrum

9.1 Organisation des sprints

Le projet est organisé en **6 sprints d'1 mois** de Février à Août 2026.

Sprint	Mois	Statut	Livrables principaux
1	Février 2026	OK	CDC, conception UML, architecture, setup
2	Mars 2026	OK	Upload CV, extraction texte, OCR, BDD
3	Avril 2026	OK	Pipeline NLP spaCy, extraction skills
4	Mai 2026	En cours	BGE-M3 + Cross-Encoder, matching, dashboard
5	Juin 2026	TODO	Sécurité, tests intégration, notifications
6	Juil–Août 2026	TODO	Déploiement Railway, documentation, soutenance

9.2 Outils utilisés

Git (GitHub), VS Code, PyCharm, Postman (tests API), Trello (backlog). Reviews régulières avec Mme Sirine Nasri (encadrante société) et Mme Loujayne Bouzarti (encadrante académique).

Chapitre 10

Avancement actuel — Sprint 4

10.1 Fonctionnalités livrées

Authentification complète (JWT, OAuth Google/LinkedIn, OTP email).
Upload et extraction multi-format (PDF textuel, DOCX, PDF scanné via OCR).
Pipeline NLP complet : compétences, expériences, formations, langues.
Moteur de matching BGE-M3 + Cross-Encoder + MLP Fusion.
Dashboard recruteur : liste des offres, classement des candidats.
Interface candidat : liste des offres compatibles, candidature avec RGPD.
Score détaillé par dimension dans MatchReport.
Résumé IA du profil (Claude/Mistral).
Notifications (base implémentée).

10.2 Démo — Scénario de test

Comptes de démo

- Recruteur : `selim@esprit.tn` / `Demo3S2026!`
- Admin : `admin1@esprit.tn` / `Demo3S2026!`
- Offre de démo : **Developpeur React Senior**
- 8 candidats avec scores de 93.7% (Ahmed Benali) à 5.3% (Amina Diallo)

Chapitre 11

Perspectives

1. **Notifications email recruteur** — alerte à chaque nouvelle candidature.
2. **Planification d'entretiens** — créneaux proposés par le recruteur, acceptés par le candidat.
3. **Page candidat enrichie** — compétences manquantes identifiées, conseils personnalisés.
4. **Amélioration continue par feedback** — le recruteur note « embauché / refusé », ces données alimentent un réentraînement du MLP.
5. **Génération de rapports via LLM (Mistral AI)** — rapport narratif complet sur chaque candidat.
6. **Déploiement Railway** — conteneurisation Docker, CI/CD GitHub Actions.

Conclusion

TalentMatch apporte une réponse technique complète à la problématique du matching CV / offres d'emploi. Le pipeline IA (BGE-M3 + Cross-Encoder + MLP Fusion) dépasse les approches par mots-clés des solutions existantes en comprenant la **sémantique** des profils et des offres.

Les choix technologiques (FastAPI, PostgreSQL, React, spaCy) ont été guidés par des critères clairs : performance, maintenabilité et adéquation avec le périmètre du projet. L'architecture modulaire (*pipeline-shaped*) facilite les évolutions futures.

Le Sprint 4 marque le cœur du projet avec la livraison du moteur de matching IA. Les sprints 5 et 6 compléteront la solution avec la sécurité renforcée, les tests d'intégration et le déploiement en production sur Railway.

Youssef Gara — Stage PFE 3S TalentMatch — Juin 2026